

---

```
>>> letters = ['z', 'A', 'a', 'Z']
>>> letters.sort()
>>> letters
['A', 'Z', 'a', 'z']
```

---

The American Standard Code for Information Interchange (ASCII, pronounced “ask-ee”) is a mapping between numeric codes (called *code points* or *ordinals*) and text characters. The `sort()` method uses *ASCII-betical* sorting (a general term meaning sorted by ordinal number) rather than alphabetical sorting. In the ASCII system, *A* is represented by code point 65, *B* by 66, and so on, up to *Z* by 90. The lowercase *a* is represented by code point 97, *b* by 98, and so on, up to *z* by 122. When sorting by ASCII, uppercase *Z* (code point 90) comes before lowercase *a* (code point 97).

Although it was almost universal in Western computing prior to and throughout the 1990s, ASCII is an American standard only: there’s a code point for the dollar sign, \$ (code point 36), but there is no code point for the British pound sign, £. ASCII has largely been replaced by Unicode, because Unicode contains all of ASCII’s code points and more than 100,000 other code points.

You can get the code point, or ordinal, of a character by passing it to the `ord()` function. You can do the reverse by passing an ordinal integer to the `chr()` function, which returns a string of the character. For example, enter the following into the interactive shell:

---

```
>>> ord('a')
97
>>> chr(97)
'a'
```

---

If you want to make an alphabetical sort, pass the `str.lower` method to the `key` parameter. This sorts the list as if the values had the `lower()` string method called on them:

---

```
>>> letters = ['z', 'A', 'a', 'Z']
>>> letters.sort(key=str.lower)
>>> letters
['A', 'a', 'z', 'Z']
```

---